APS · DESENVOLVIMENTO DE SISTEMAS DISTRIBUÍDOS

ViaAgora

O tempo de espera das balsas da Baixada Santista, em tempo real, no seu celular



Bruno Nascimento de P. Silva

RA G793030

UNIP · Ciência da Computação · Santos, 2026

O problema e o ViaAgora

A espera muda o tempo todo

Pico, maré baixa e neblina mudam a fila nas três travessias

O painel oficial é limitado

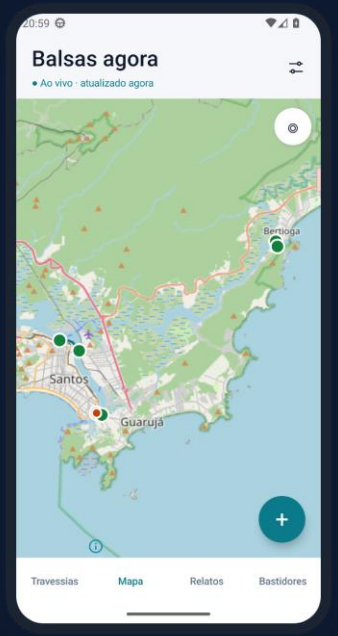
Sem histórico, sem avisos e sem a visão de quem está na fila

O ViaAgora junta tudo, ao vivo

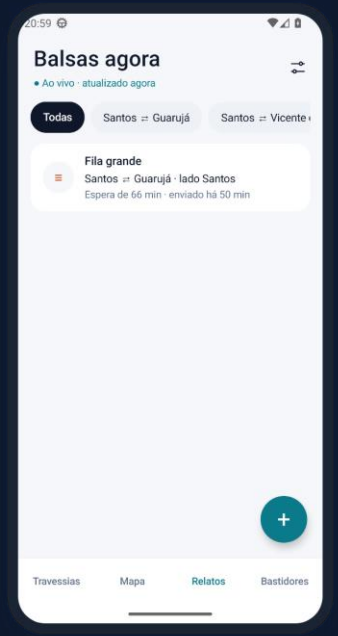
Dado oficial + relatos da comunidade, avisados na hora a todos os celulares



Travessias



Mapa



Relatos

Mas o principal está por trás das telas: um sistema distribuído.

O que é um sistema distribuído

“Um conjunto de computadores independentes que se apresenta aos usuários como um sistema único e coerente.”

Tanenbaum e Van Steen (2007)



Concorrência

Vários processos trabalham ao mesmo tempo

No ViaAgora: Dois celulares enviam relatos juntos, para réplicas diferentes



Sem relógio global

Cada máquina tem o seu relógio

No ViaAgora: Os tempos dos eventos usam o relógio do banco como referência

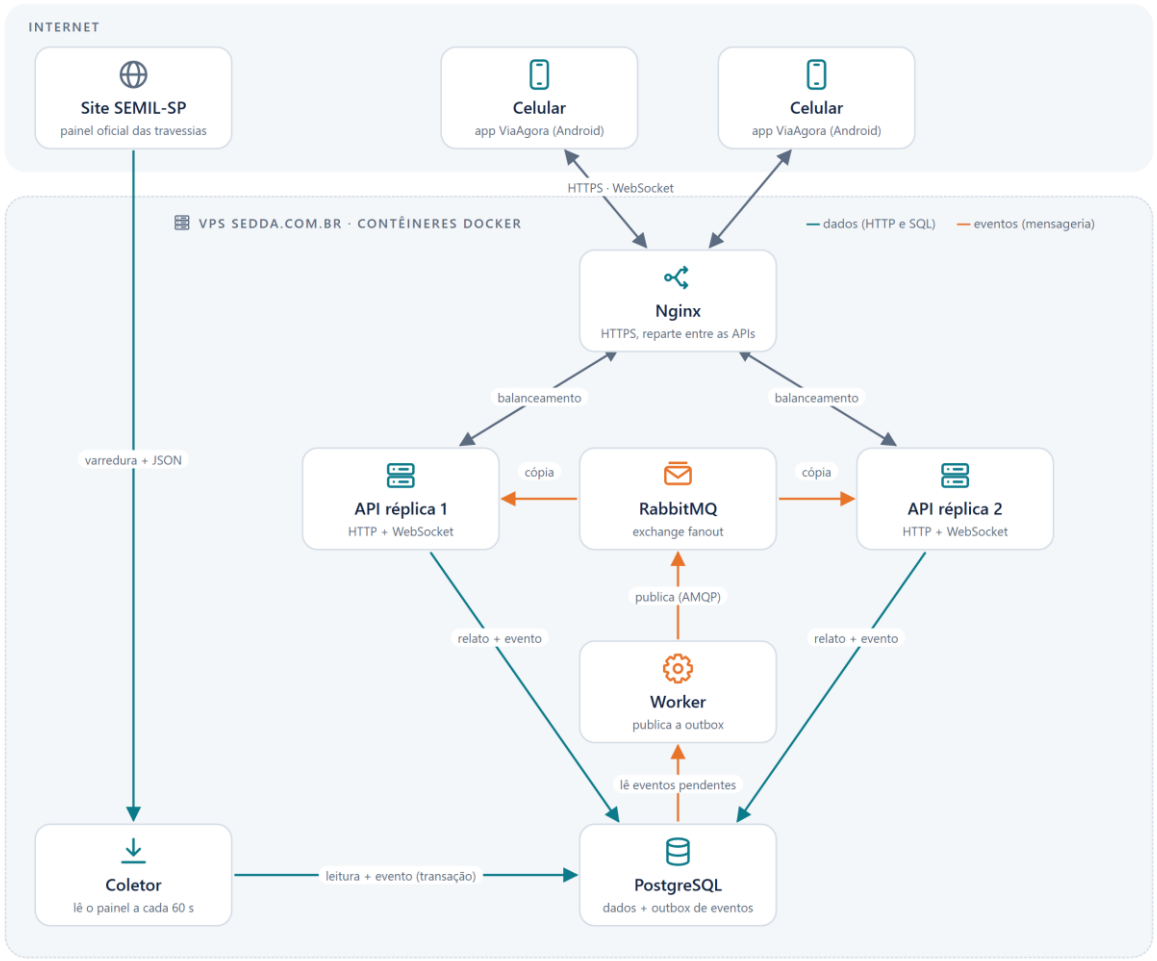


Falhas independentes

Uma parte cai, as outras continuam

No ViaAgora: O broker cai e a API segue salvando os relatos

A arquitetura do ViaAgora



7 processos independentes, em contêineres Docker, que só conversam pela rede.

- **Nginx** Única entrada; reparte os celulares
- **2 réplicas da API** Mesmo código, mesmo banco
- **Worker + RabbitMQ** Levam os eventos às réplicas
- **PostgreSQL** Fonte da verdade e outbox
- **Coletor** Lê o painel oficial a cada minuto

Comunicação entre os processos



HTTP

SÍNCRONA

O celular pede e espera a resposta: consultar travessias, enviar um relato.



WebSocket

CONEXÃO ABERTA

O servidor avisa o celular na hora, sem o celular ficar perguntando.



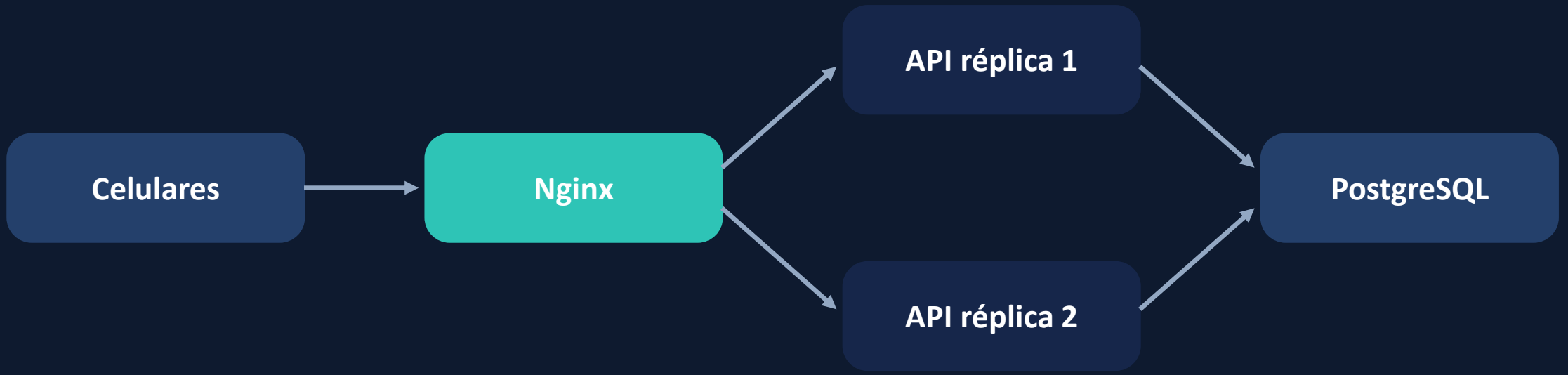
AMQP

ASSÍNCRONA

O worker publica e cada réplica recebe quando puder; ninguém espera ninguém.

A ideia: o que precisa de resposta imediata é síncrono; o que pode chegar depois vai por mensagem, sem prender os processos.

Replicação e balanceamento



Duas cópias iguais

Mesmo código e mesmo banco: qualquer uma atende qualquer celular.

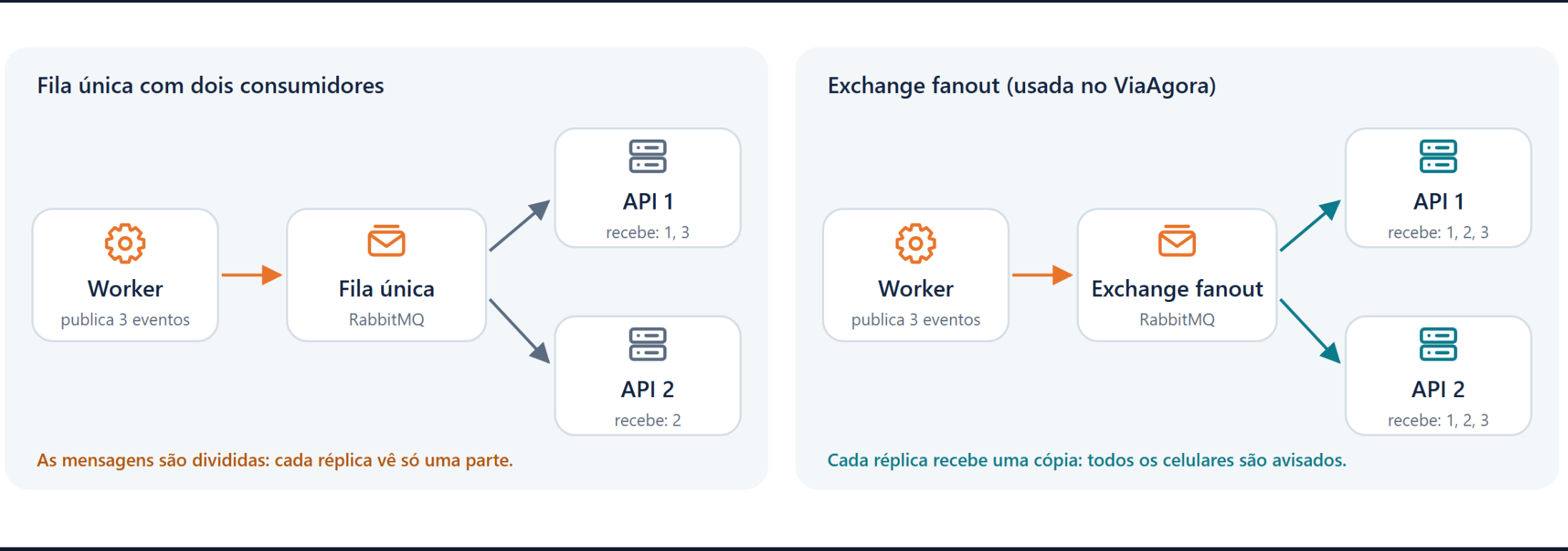
O Nginx alterna

Cada pedido vai para uma réplica, em rodízio.

Uma cai, a outra assume

Sem resposta em 2 s, o Nginx manda tudo para a outra.

Mensageria: publish/subscribe



Por que fanout? cada réplica tem os seus próprios celulares ligados. Se as mensagens fossem divididas, metade dos usuários não seria avisada. Com a exchange fanout, todas as réplicas recebem todos os eventos.

Consistência: nenhum relato se perde

Celular envia

Relato + evento na mesma transação

Worker publica e espera a confirmação

Cópia para cada réplica

Celulares avisados e buscam os dados

Outbox transacional

O evento é gravado no banco junto com o relato. Se o RabbitMQ estiver fora do ar, o evento espera e sai depois.

Idempotência

Cada envio tem uma chave. Se a rede falhar e o app tentar de novo, o servidor reconhece e não duplica o relato.

Quando algo falha

O RabbitMQ cai

Os relatos continuam sendo salvos; o evento espera na outbox e sai quando o broker volta.

Outbox transacional

Uma API cai

O Nginx deixa a réplica parada de lado em até 2 s e manda todos para a outra.

Replicação + balanceamento

O painel oficial cai

O coletor tenta de novo esperando mais a cada vez; o app mostra o último dado como atrasado.

Backoff

O celular perde a conexão

O app reconecta e busca tudo de novo; a cada 30 s também consulta a API.

Ressincronização

Transparência: o usuário não percebe a distribuição

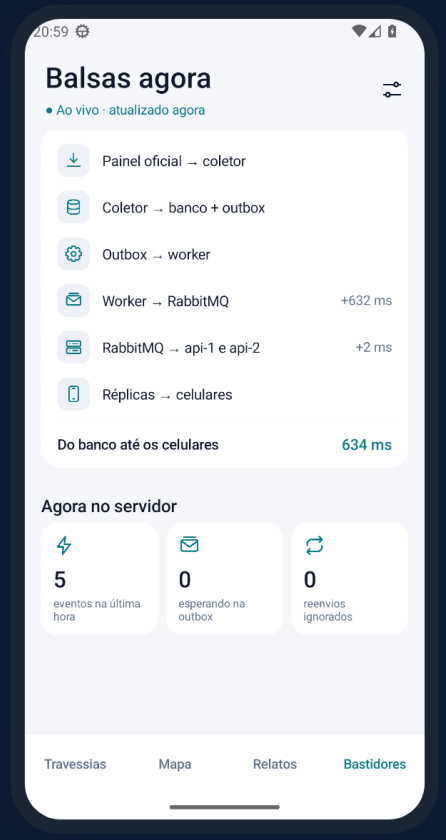
Acesso e localização	O app fala com um único endereço (sedda.com.br); não sabe onde cada processo roda.
Replicação	O usuário não sabe se foi atendido pela réplica 1 ou pela 2.
Falha	Uma réplica ou o broker caem e o app continua funcionando.
Concorrência	Vários celulares usam o sistema ao mesmo tempo sem atrapalhar uns aos outros.

Tipos de transparência segundo Tanenbaum e Van Steen (2007).

Bastidores: o sistema visível no app



Diagrama ao vivo



Caminho do evento

Cada quadro é um processo

Com a bolinha verde do sinal de vida (heartbeat)

Você no desenho

O seu celular e a réplica a que ele está ligado aparecem destacados

O evento anda na tela

Cada mensagem é animada, com o tempo medido de cada etapa

Modo passo a passo

Para explicar o caminho de um evento em aula

Demonstração ao vivo

1. Enviar um relato num celular e ver chegar no outro
2. Acompanhar o evento na aba Bastidores
3. Parar o RabbitMQ: o relato espera na outbox
4. Parar uma API: o Nginx passa tudo para a outra



Baixe o app

sedda.com.br



Código no GitHub

Bruno70i/APS---8-SEMESTRE

Roda em qualquer computador: `docker compose up --build -d`

Obrigado!

Perguntas?

sedda.com.br

github.com/Bruno70i/APS---8-SEMESTRE

Bruno Nascimento de P. Silva

RA G793030

